OpenLL: an API for Dynamic 2D and 3D Labeling

Daniel Limberger*, Anne Gropler, Stefan Buschmann, and Jürgen Döllner†
Hasso Plattner Institute, University of Potsdam
Potsdam, Germany
Email: {daniel.limberger, juergen.doellner†}@hpi.de*

Benjamin Wasty‡
Seerene
Potsdam, Germany
Email: benjamin.wasty@seerene.com‡

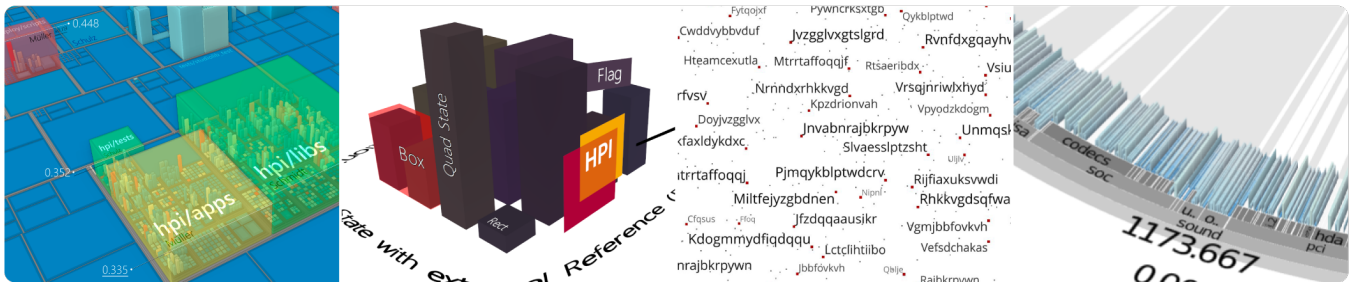


Figure 1. fltr, a data visualization using interactive annotation, a synthetic scene showcasing automatic label placement, a benchmark for point-feature label placement, and a circular bundle view using labeling with adaptive level-of-detail, automated label orientation as well as text-alignment along B-splines.

***Abstract*—Today's rendering APIs lack robust functionality and capabilities for dynamic, real-time text rendering and labeling, which represent key requirements for 3D application design in many fields. As a consequence, most rendering systems are barely or not at all equipped with respective capabilities. This paper drafts the unified text rendering and labeling API OpenLL intended to complement common rendering APIs, frameworks, and transmission formats. For it, various uses of static and dynamic placement of labels are showcased and a text interaction technique is presented. Furthermore, API design constraints with respect to state-of-the-art text rendering techniques are discussed. This contribution is intended to initiate a community-driven specification of a free and *open label library*.**

***Keywords*-visualization, labeling, real-time rendering**

I. Introduction

"A major limiting factor to the widespread use of information visualization is the difficulty of labeling information abundant displays" [1]. This assessment of Fekete and Plaisant is as tenable today as it was in 1999. Most of today's visualizations are still troubled by missing labels, leading to one-by-one assessment of graphical elements, overlapping labels obscuring depicted data, and ambiguous association of labels to graphical elements. Even recent advances in big-data visualization have not managed to resolve these well-known labeling issues, despite their tremendous potential for increasing the visualization quality.

We suppose there are at least two reasons for labeling to remain mostly unattended in real-time computer graphics. First, high-quality text rendering and placement is an intricate task, especially in 3D. It starts with text rendering itself requiring lots of implementation and use-case specific tweaking and probably ends with having no text rendering support in any of the latest rendering APIs, be it OpenGL [2], WebGL [3], or Vulkan [4]. In research, this means lots of available resources ought to be allocated for prototype development rather than novelty and evaluation. Second, other industries, like the gaming industry usually get along with basic (bitmap based) text rendering. With a few exceptions (e.g., *Tom Clancy's The Division*, developed by Massive Entertainment and published by Ubisoft in 2016, shows impressive dynamic labeling in its UI and world map), high-end labeling is attributed to artistic rather than programming efforts and is the result of tweaking and pre-computing assets in compliance with a title's art direction. Furthermore, the number of labels is typically much smaller and sophisticated placement requirements are not present. Accordingly no industry seems to drive the progression or standardization of labeling technology in general.

In order to initiate a research driven specification of an open label library, we (1) put our idea of a unified labeling API forward for discussion, and (2) propose subsequent steps required for a widespread use in information visualization. To this extent, the following contributions are made:

- a brief introduction to and overview of existing text rendering approaches and systems is provided,
- snippets of the labeling API *OpenLL*, derived from use cases of our research group, are presented, and

- our ongoing open-source implementation efforts for mobile and desktop applications are described.

OpenLL is designed to complement existing rendering APIs, systems, and transmission formats (such as glTF [5]). It facilitates a descriptive labeling approach using label and proxy objects that are processed by means of renderers that are aligned to implementation specifics of hardware-accelerated text rendering techniques. Thus, this paper provides the general concepts of OpenLL instead of drafting an extension to any rendering API or transmission format.

II. Related Work

This section covers related work in the fields of text rendering techniques and systems, and labeling in information visualization in general.

A. Text Rendering Techniques

With OpenLL we are targeting real-time rendering scenarios in which any additional processing might impact the overall rendering performance or interaction experience. This means that rendering hundreds of labels within a virtual reality application at 90Hz or higher, for example, constrains the available processing time for labeling within each frame to at the very most a millisecond. Accordingly, the best practice for increasing real-time rendering performance is often to "avoid text rendering whenever possible"[1]. To counter this practice, OpenLL is specifically aligned to text rendering techniques that take advantage of texture-mapping graphics hardware by using bitmaps, signed distance fields, and vector maps.

Bitmap font rendering is the basis for rendering dynamic text, i.e., text that can change at run-time and cannot be prerendered into a texture. The text is divided into its elemental symbols (*glyphs*), each represented by a quad and typeset (using additional kerning data or mono space) on the CPU. Each glyph's coverage values are prerendered into a bitmap using, e.g., FreeType [6] or quadratic curve evaluation [7], and packed into a texture map. The GPU is then used to texture every glyph using its respective subregion within the texture map. This approach is feasible as long as the text is displayed at a fixed size in pixels.

Signed distance fields (*SDF*) representing the glyphs' shapes can be used to better account for varying display resolution as well as text size and orientation [8]. Each distance function is sampled and encoded as a bitmap within the texture map. The GPU can then reconstruct a glyph's shape at any orientation by interpolating bilinearily and thresholding the distances. Furthermore, per-pixel screen-space derivatives can be used to properly anti-alias the glyph edges [9] and multiple texture channels, representing different edges intersecting within a texel, can be used in order to preserve sharp corners [10].

Spline fragment-based sampling approaches are recently considered to fully overcome all limitations of the bitmap and SDF based approaches. Quadratic Bézier curve segments for every glyph are efficiently encoded in a texture. This data is then decoded and locally super sampled per fragment on the fly [11]. "It should be clear that [this approach] requires considerably more computation" [12] (and dependent texture lookups) in comparison to SDF. However, this approach is still fast enough to enable rendering of many labels in real-time while providing the most flexibility and best quality.

B. Text Rendering Systems

The presented API requires text to be placed and transformed in virtual two and three dimensional scenes. Text rendering facilities exposed via native GUI APIs such as *Core Text*, *Direct Write*, Windows *GDI*, *Qt*, *X11*, or *Xft* are usually tightly integrated into a framework or operating system. For the most part, though supporting hardware acceleration, they cannot be used from within custom hardware accelerated contexts nor be non-trivially transformed.

Font rendering libraries such as FreeType [6] are focussing on handling font faces, typesetting, and text rendering in 2D and can be used, e.g., to prerender glyphs to textures for hardware accelerated use. One promising library for fast text rendering using OpenGL is *freetype-gl* [13]; however, there are no means for advanced placement and arbitrary transformations yet.

For web applications targeting browsers there is a similar lack of support. Even though browsers have advanced text rendering capabilities, they are not fully exposed to developers. The rendering library *three.js* [14] creates vertex-based geometry for text rendering (there are community examples showing SDF rendering that use three.js's text capabilities though) and suggests HTML or canvas rendering for the visual display of large amounts of text. With CSS 3 and HTML one could achieve adequate 2D and 3D transformed labeling by creating a pool of html text elements and changing their position and content dynamically [15]. This, however, does not allow for occlusion by the 3D scene (e.g., shared depth buffer) and yields inefficient rendering performance.

C. Labeling in Information Visualization

OpenLL is intended as an extensible API covering various labeling approaches. Kakoulis and Tollis provide a good introduction and overview to labeling problems that were identified within the context of cartography and rendering maps [16]. Labeling techniques can be categorized as internal or embedded [17], external, boundary, and excentric labels [18]. For their placement, various quality measures and requirements are commonly applied: (1) readability w.r.t. overlap, orientation, and occlusion with other graphical

[1]Performance tips for using the canvas: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial/Optimizing_canvas

elements, (2) unambiguity w.r.t. to the referenced/associated graphical element(s), (3) minimization of visual clutter, (4) elegibility for dynamic placement in real-time, (5) temporal-coherency by means of reducing visual discontinuities, and, finally, their (6) compaction, i.e., the minimization of each individual layout area itself [19]. In addition, problem and computation specific measures and constraints, e.g., attractive and repulsive forces [20] might be introduced.

A vast number of labeling placement techniques such as positioning names on maps [21], point feature label placement [22], external label management in 3D object space [23], or force-based labling [24] have been introduced. Furthermore, the labels themselves can be used as proxies for interaction as shown by, e.g., Balata et al. [20]. For the design of OpenLL this necessitates extension points that account for and embrace the diversity of application domains and labeling approaches.

III. Labeling Model

The core of our labeling API is about management and automated placement of a large number of labels in 2D and 3D in real-time. It does not provide implementation specifics, e.g., for computational math, parallelization, or hardware accelerated rendering, nor does it inflict a certain implementation design. However, for deriving font handling interfaces and general rendering features it is relevant to account for common limitations of today's rendering approaches. Similar to rendering APIs we expose objects and operate on the object relations and states. For it, we introduce the following API objects:

Font Face — The font face describes the resources required for rendering text. This includes, e.g., a list of available glyphs and ligatures, their orientation, advance, weight, and kerning data, their location within a texture map as well as the texture map's encoding, e.g., 16bit floating point SDF samples or 8bit RGBA coverage. We assume all assets to be generated beforehand, but a font family reference could also be used if the OpenLL driver is capable of generating the required assets on the fly.

Text — This object stores the actual text by means of a dynamic sequence of characters or code points.

Label — A label combines all data required for typesetting, positioning, orienting, and rendering text. It references a text or an excerpt of a text, a font face and style definition, and describes its specific placement via positions, margins, anchors, proxies, constraints, or auxiliary shapes such as lines, points, etc. It also represents the main extension point for domain specific labeling techniques.

Label Cloud — This object can be used for data driven labeling, e.g., when large number of points or objects are selectively labeled based on weights or other label-specific data. For that, the text references, placement and weight data must be provided. The font face and style definitions are referenced only once, and depending on the cloud's specialization, additional data such as a label selection description, maximum number of labels to be displayed, or no-label regions can be added.

Proxy — Similar to real-time collision physics, implicit shapes like spheres, cubes, quads, points, lines, or splines can be used to partially approximate detailed 2D and 3D scenes or describe dynamic anchors such as a virtual camera or an area of interest. With this, the computational complexity of label placement and anchoring, especially for more advanced, e.g., constraint-based, adaptive label placement techniques can be reduced. Any Label can then reference a proxy's vertices, edges, or areas as an anchor. For example, a cuboid proxy could expose each of its corners, edges, center points, or even its lateral surfaces for anchoring. Proxies also provide the basis for detecting overlapping labels, adapting to proxies occluding labels, or using their extent for level-of-detail control.

Renderer — The renderer is tasked with configuration, preparation and real-time rendering of a given set of labels or label clouds. There might be various rendering implementations favoring specific text rendering approaches or font assets. OpenLL does not assume any rendering implementation details, and instead provide, if any, generic quality and performance hints, e.g., flags for favoring quality over memory. It is the renderer's responsibility to internally gather data such as a device pixel ratios for correct font size scaling and display sub-pixel layout for sub-pixel anti-aliasing. Finally, the renderer must allow label retrieval via screen space coordinates for user interaction purposes.

Using these objects, the general approach for rendering labels is as follows: (1) prepare font faces and proxies, (2) instantiate labels or label clouds, (3) configure one or multiple renderer and callbacks, and, finally, (4) update texts and proxies and trigger the rendering on every frame. The actual placement of the labels should be triggered by the renderer internally in the following manner: all labels and clouds that are about to be rendered are triggered for a lazy update. If any of their properties or referenced objects (font face, proxies, etc.) has changed, all required transformations for placement and rendering are (re)computed and associated geometry (re)submitted for rendering.

Overall, this design facilitates a community driven standardization of 2D and 3D labeling by means of extensions, facades, or new libraries, and thus promotes the use of existing text rendering engines, e.g., *glyphy* [25], *freetype-gl* [13], *FreeType* [6], or even native frameworks.

IV. Labeling API

This section discusses the labeling API by examples derived from use cases in the fields of Geovisualization and Software visualization. The examples are in pseudocode, but the syntax is inspired by TypeScript for readability.

A. Basic Label Placement in 2D

The first listing shows how to create and render a single 2D label, specifying basic font and placement options:

```
1
2  const fontFace = FontFace.fromFile(
3      'opensans.regular.fnt');
4
5  let label = new Position2DLabel({
6      text: 'Hello Word!',
7      fontFace: fontFace,
8      fontSize: { value: 12.0, unit: 'pt' },
9      alignment: 'left',
10     position: { x: 32, y: 8, unit: 'px' },
11     direction: { x: 0.0, y: 1.0} });
12 label.lineAnchor = 'baseline';
13
14 var renderer = new GlyphRenderer();
15 renderer.superSampling = 'rgss';
16 renderer.draw(label);
17
```

The options can be specified declaratively by passing a JSON object to the label constructor, but also changed procedurally later (line 12). They are loosely based on CSS rules where possible. For example, the font face can be specified as a relative path to a file. There are also options for the renderer that can be set dynamically, such as supersampling using RGSS (*rotated grid supersampling*).

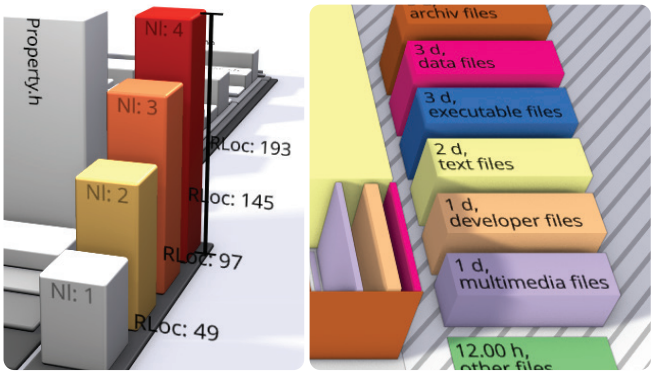
B. Multi-line Labels in 2D

This example shows a multi-line label that is rendered into a given rectangle, specifying word wrap, overflow behavior, and padding in a similar manner to CSS. This allows, e.g., for labeling proxy objects such as faces of a bounding box:

```
1
2  let label = new Rectangle2DLabel();
3
4  label.fontSize = { value: 12, unit: 'pt' };
5  label.rectangle = { x: 0, y: 0,
6      width: 512, height:128, unit: 'px' };
7  label.wordWrap = true;
8  label.padding = { bottom: 8, left: 8,
9      top: 8, right: 8, unit: 'px' };
10 label.lineHeight = { value: 16, unit: 'pt' };
11 label.overflow = { action: 'hidden',
12     per: 'line' };
13
```

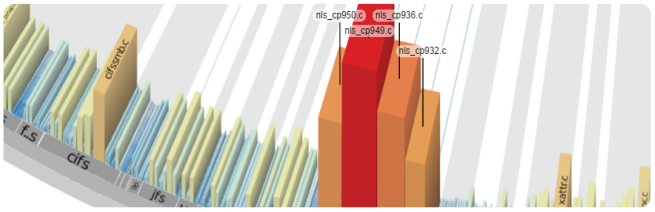
C. Label at 3D Position

This is a basic example of static label placement in 3D space. Contrary to 2D space, we cannot use pixel or point as a unit for font size and positioning, since the actual size of a label depends on its position and the position of the camera. Thus there is no purpose in using any unit other that world space coordinates for font size and positioning:




```
1
2  let label = new Position3DLabel();
3  label.fontSize  = 0.125;
4  label.alignment = 'right';
5  label.position  = { x: 0.0, y: 0.0, z: 0.0};
6  label.direction = { x: 1.0, y: 0.0, z: 0.0};
7  label.up        = { x: 0.0, y: 0.0, z: 1.0};
8
```

D. 2D Projected Label at 3D Position



Projected 3D labels are positioned in 3D space, but projected onto a 2D plane. The model view projection is used to position the label based on its anchor point that is specified in world coordinates, while the label itself is treated as 2D:

```
1
2  let label = new Projected3DLabel();
3  label.fontSize  = { value: 12, unit: 'pt' };
4  label.position = { x: -0.05, y: 0.02, z: -0.3 };
5  label.direction = { x:  0.0,  y: 1.0 };
6
7  let renderer = new GlyphRenderer();
8  renderer.modelViewProjection = mvp;
9  renderer.draw(label);
10
```

E. Labeling along 3D Splines



Splines can be useful, e.g., for annotating a 3D-building by wrapping a label around the walls or for labeling a river. A spline has several properties: control points, knot values and degree. Two of them must be given for defining the spline, the third can be calculated. For every glyph on a spline, the same up vector can be used. However in 3D space, the up vector can also be different for every glyph. In addition to the base spline (that is the baseline in typesetting), a top spline is defined, which is used to calculate an up vector for every glyph. That up vector is defined by the position of the glyph on the base spline and the corresponding position on the top spline:

```
let label = new Spline3DLabel();
label.position = { x: 0.0, y: 0.0, z: 0.0};

controlPoints = [
    { x: -1, y: 0 z: 1}, { x: -1, y: 0 z: 0},
    { x: -1, y: 0 z: 0}, { x: -1, y: 0 z: 1} ];
controlPoints2 = [
    { x: -1, y: 1 z: 1}, { x: -1, y: 1 z: 0},
    { x: -1, y: 1 z: 0}, { x: -1, y: 1 z: 1} ];
knotValues = [ 0.f, 0.f, 0.f,
    1.f, 2.f, 3.f, 3.f, 3.f ];
degree = 3;

label.baseSpline = new Spline3D(
    controlPoints, degree, knotValues);
label.topSpline = new Spline3D(
    controlPoints2, degree, knotValues);

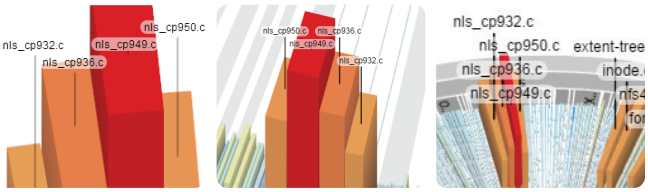
label.additionalTransform = matrix;

```

F. Adaptive Label Placement

To maintain readability for rendered glyphs, their placement should adapt to changes of view or occluding geometry. When annotating a point, the label can be placed at different angles; when annotating a line, the label can be written over any part of the line – to mention a few scenarios. We address those issues with two examples, based on uses cases in Treemap-based software visualizations:

1) Adaptive Label Placement in Screen Space: The first example shows an external label in screen space. It uses a line in world space to connect the label to the corresponding object. Changing the view might later result in changing the line origin and thus changing the label position.



2) Adaptive Label Placement using a Box Proxy: The second example uses a proxy in form of a box. Different surfaces of the box are usable for the label's position. It also



possible to change the direction and up vector of the label accordingly. At this point, a smart placement algorithm can be interposed to calculate the label's placement taking into account the proxy geometry, the current view, or occluding geometry:

```

let box = new BoxProxy();
box.extents = {
    widthSpan: { x: 2.0, y: 0.0, z: 0.0 },
    heightSpan: { x: 0.0, y: 0.8, z: 0.0 },
    depthSpan: { x: 0.0, y: 0.0, z: 0.5 },
};
box.position = { x: 0.0, y: 0.0, z: 0.0 };

let label = new Position3DLabel();
label.fontSize = 0.125;
label.position = box.upperSurface;

```

G. Labeling with Level Of Detail



When rendering a large amount of labels, the view is easily cluttered and labels overlap or occlude other objects. To prevent that, it is possible to choose a subset of all labels based on certain constraints, e.g., only the 20 most important labels (criterion: label priority or weight), a minimum font size (view-based in 3D), or only certain semantic groups that can have precedence for visual display over other groups.

The following example shows how to define such a label cloud, where multiple labels with the same positioning and stylization properties are gathered. Constraints of different types are added that are evaluated before the labels are rendered:

```

let cloud = new LabelCloud();
cloud.texts = ['label1', 'label2',
                ..., 'labelN'];

cloud.positions = [
    { x: -0.0134, y: 0.0911, z: -0.2184 },
    { x: -0.0267, y: 0.0526, z: -0.3926 }, ...
    { x: -0.0589, y: 0.0139, z: -0.9424 } ];

cloud.weights = [ 0.2, 0.3, ... 0.1 ];
```

```
cloud.direction = { x: 0.0, y: 1.0 };
cloud.algorithm = 'SimulatedAnnealing';
cloud.alignment = 'left';

cloud.constraints.add({
    type: 'GlobalMaxLabels',
    value: 30 });
cloud.constraints.add({
    type: 'GroupMaxLabels',
    value: 10,
    criterion: weight > 0.1 });
renderer.draw(cloud);

```

V. Towards an Open Labeling Library

The fast and wide-spread adoption of Vulkan [4] and glTF [5] are two recent examples for successful introductions of free and open standards. To address industry and community needs, it is crucial not only to provide plain specifications, but to set up infrastructural means that facilitate adoption by enabling developer feedback, and providing comprehensive documentation, design guidelines and lots of examples. Most importantly though, we consider the availability of free and open-source reference implementations as well as tools for font-asset generation and API validation to be crucial for a successful adoption of an open label library.

Though not directly intended as an extension to a specific transmission format or rendering API, the presented labeling library is designed to fit today's ecosystem of graphic rendering APIs and real-time rendering techniques. It should provide reference implementations targeting various rendering APIs from the start. We are currently focusing on a TypeScript-based WebGL as well as a C++ based OpenGL implementation, but are committed to subsequent ports targeting OpenGL ES and Vulkan in the future. Both renderers are capable of layouting and rendering thousands of glyphs within a fraction of a millisecond using one of multiple rendering backends (bitmap, SDF, and spline fragments).

As of writing this paper, there is a limited number of robust font asset formats and generators available. Our reference implementation relies on the quasi-standard bitmap font format[2] which can be easily extended. In order to generate font assets we used well known font generators such as *Hiero* or *Littera* (deprecated due to Adobe Flash). Due to several shortcomings, we started development of an extensible, free and open-source font-asset generator that features

- interactive text rendering preview in 2D and 3D for efficient iterative font asset refinement,
- C++ implementation for fast texture packing, signed distance field computation and glyph rasterization (Figure 3),
- use via command line interface, via integration as library, or via its graphical user interface on all major platforms.

[2]http://www.angelcode.com/products/bmfont/

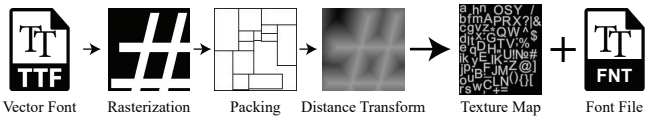


Figure 3. Processing pipeline of our font asset generation library.

VI. Conclusion and Future Work

In this paper we presented a draft for an open label library, *OpenLL*, an API that is designed to complement existing rendering APIs, frameworks, and transmission formats. To this end the API provides key objects, i.e., proxies and labels, for configuration and management of labels in 2D and 3D in real-time. Proxies can be used to approximate a virtual scene and provide various anchors that are then referenced by labels for placement. The label itself provides (1) a detailed description of the font face and the actual text for rendering, and (2) the API's main extension point for various types of label placement techniques. In order to facilitate adoption and community feedback, the API design does not limit the text rendering driver to a specific set of techniques, even though the OpenLL is intended to be used in interactive rendering applications. Our free and open-source resources [26], as partially presented, comprises (1) two reference implementations of the depicted API examples using WebGL and OpenGL and (2) a cross-platform font asset generator.

Both our implementations currently do not support ligatures, composite code points, or more sophisticated typesetting. Furthermore, we would like OpenLL to support extruded 3D text (as in *three.js*) and CSS-like stylization options (gradient, outlines, glow, drop shadows, and more). Finally, we hope this work inspires a community effort, especially in the field of information visualization, towards creating more accessible and unified labeling systems.

Acknowledgment

The authors would like to thank Daniel Gimbatschki and Simon Krogman for their valuable API design and implementation contributions. This work was funded by the German Federal Ministry of Education and Research (BMBF) within the BIMAP project (https://www.bimap-project.de).

References

[1] J.-D. Fekete and C. Plaisant, "Excentric labeling: Dynamic neighborhood labeling for data visualization," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ser. CHI '99. New York, NY, USA: ACM, 1999, pp. 512–519. [Online]. Available: http://doi.acm.org/10.1145/302979.303148

[2] M. Segal and K. Akeley, "The opengl graphics system: A specification - version 4.6 (core profile)," https://www.khronos.org/registry/OpenGL/specs/gl/glspec46.core.pdf, 2017.

[3] Khronos Group, "Webgl 2.0 specification," https://www.khronos.org/registry/webgl/specs/latest/2.0/, 2018.

[4] Khronos Vulkan Working Group, "Vulkan 1.0.69 - a specification," https://www.khronos.org/registry/vulkan/specs/1.0/pdf/vkspec.pdf, 2018.

[5] Khronos 3D Formats Working Group, "Gl transmission format specification," https://github.com/KhronosGroup/glTF, 2017.

[6] Turner, David and Wilhelm, Robert and Lemberg, Werner, "Freetype," https://www.freetype.org/index.html, 1996.

[7] E. Wallace, "Easy scalable text rendering on the gpu," https://medium.com/@evanwallace/easy-scalable-text-rendering-on-the-gpu-c3f4d782c5ac, 2016.

[8] S. F. Frisken, R. N. Perry, A. P. Rockwood, and T. R. Jones, "Adaptively sampled distance fields: A general representation of shape for computer graphics," in *Proceedings of the 27th Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH '00. New York, NY, USA: ACM Press/Addison-Wesley Publishing Co., 2000, pp. 249–254. [Online]. Available: http://dx.doi.org/10.1145/344779.344899

[9] Z. Qin, M. D. McCool, and C. S. Kaplan, "Real-time texture-mapped vector glyphs," in *Proceedings of the 2006 Symposium on Interactive 3D Graphics and Games*, ser. I3D '06. New York, NY, USA: ACM, 2006, pp. 125–132. [Online]. Available: http://doi.acm.org/10.1145/1111411.1111433

[10] C. Green, "Improved alpha-tested magnification for vector textures and special effects," in *ACM SIGGRAPH 2007 Courses*, ser. SIGGRAPH '07. New York, NY, USA: ACM, 2007, pp. 9–18. [Online]. Available: http://doi.acm.org/10.1145/1281500.1281665

[11] W. Dobbie, "Gpu text rendering with vector textures," http://wdobbie.com/post/gpu-text-rendering-with-vector-textures/, 2016.

[12] E. Lengyel, "Gpu-centered font rendering directly from glyph outlines," *Journal of Computer Graphics Techniques (JCGT)*, vol. 6, no. 2, pp. 31–47, June 2017. [Online]. Available: http://jcgt.org/published/0006/02/02/

[13] Rougier, Nicolas P., "freetype-gl," https://github.com/rougier/freetype-gl, 2011.

[14] Cabello, Ricardo, "three.js," https://threejs.org/, 2010.

[15] D. Limberger, B. Wasty, J. Trümper, and J. Döllner, "Interactive software maps for web-based source code analysis," in *Proceedings of the 18th International Conference on 3D Web Technology*, ser. Web3D '13. New York, NY, USA: ACM, 2013, pp. 91–98.

[16] K. G. Kakoulis and I. G. Tollis, "Labeling algorithms," in *Handbook of Graph Drawing and Visualization*, 2013.

[17] S. Maass and J. Döllner, "Embedded labels for line features in interactive 3d virtual environments," in *Proceedings of the 5th International Conference on Computer Graphics, Virtual Reality, Visualisation and Interaction in Africa*, ser. AFRIGRAPH '07. New York, NY, USA: ACM, 2007, pp. 53–59. [Online]. Available: http://doi.acm.org/10.1145/1294685.1294695

[18] S. Oeltze-Jafra and B. Preim, "Survey of labeling techniques in medical visualizations," in *Proceedings of the 4th Eurographics Workshop on Visual Computing for Biology and Medicine*, ser. VCBM '14. Aire-la-Ville, Switzerland, Switzerland: Eurographics Association, 2014, pp. 199–208. [Online]. Available: http://dx.doi.org/10.2312/vcbm.20141192

[19] K. Ali, K. Hartmann, and T. Strothotte, "Label layout for interactive 3d illustrations," *Journal of the WSCG*, vol. 13, p. 2005, 2005.

[20] J. Balata, L. Cmolik, and Z. Mikovec, "On the selection of 2d objects using external labeling," in *Proceedings of the 32Nd Annual ACM Conference on Human Factors in Computing Systems*, ser. CHI '14. New York, NY, USA: ACM, 2014, pp. 2255–2258. [Online]. Available: http://doi.acm.org/10.1145/2556288.2557288

[21] E. Imhof, "Positioning names on maps," *The American Cartographer*, vol. 2, no. 2, pp. 128–144, 1975.

[22] J. Christensen, J. Marks, and S. Shieber, "An empirical study of algorithms for point-feature label placement," *ACM Trans. Graph.*, vol. 14, no. 3, pp. 203–232, Jul. 1995. [Online]. Available: http://doi.acm.org/10.1145/212332.212334

[23] M. Tatzgern, D. Kalkofen, R. Grasset, and D. Schmalstieg, "Hedgehog labeling: View management techniques for external labels in 3d space," in *Virtual Reality (VR), 2014 iEEE*. IEEE, 2014, pp. 27–32.

[24] M. Vaaraniemi, M. Freidank, and R. Westermann, "Enhancing the visibility of labels in 3d navigation maps," *Lecture Notes in Geoinformation and Cartography (LNG&C)*, 2012.

[25] B. Esfahbod, "Glyphy: High quality glyph rendering using opengl es2 shaders," https://github.com/behdad/glyphy, 2011.

[26] Limberger, Daniel and Buschmann, Stefan, and Scheibel, Willy, "OpenLL," https://www.openll.org/, 2018.